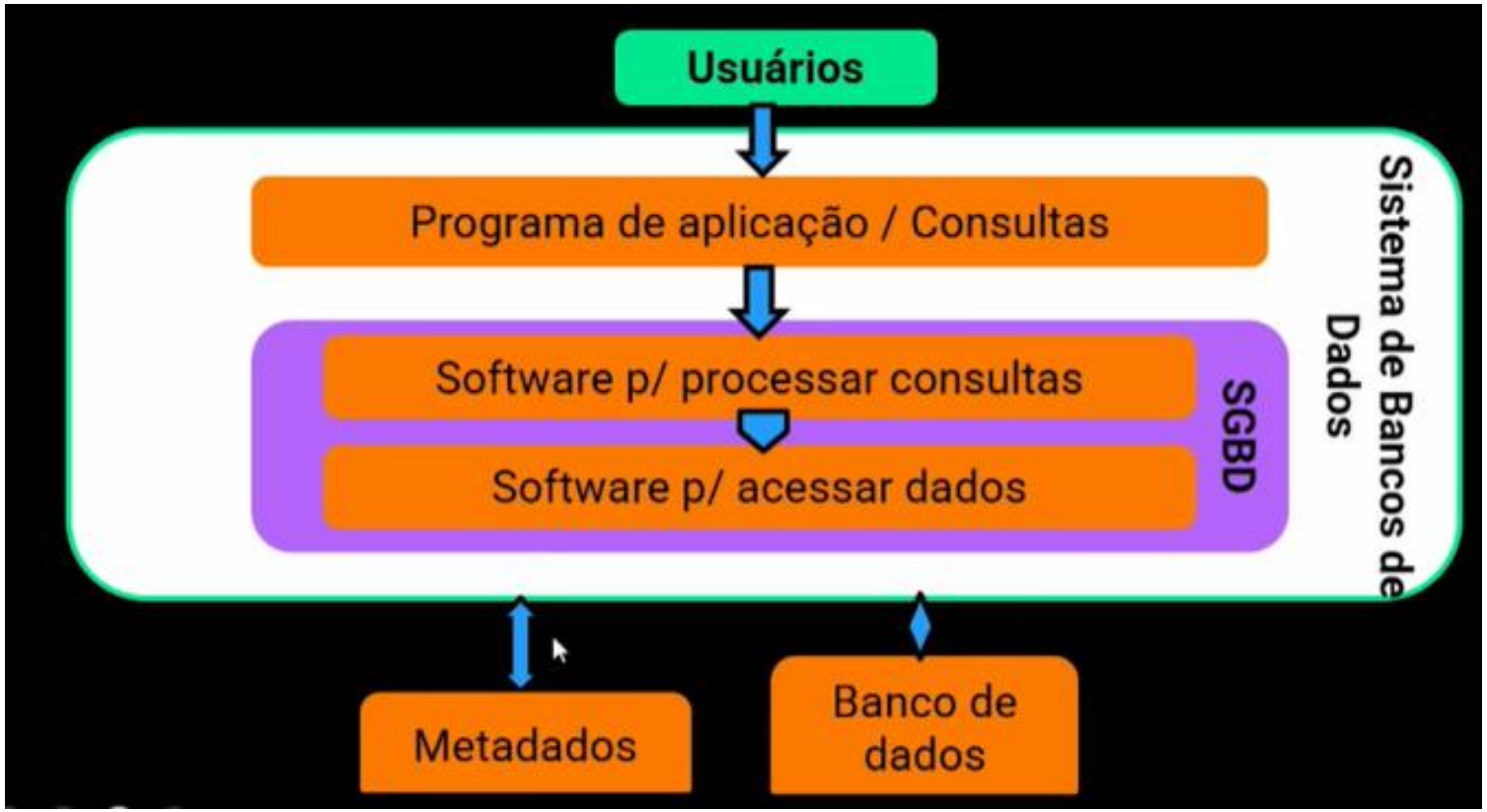


Banco de Dados

Prof. Perini

Revisão para Prova NP1

Estrutura de um SGBD



Projeto de Banco de Dados



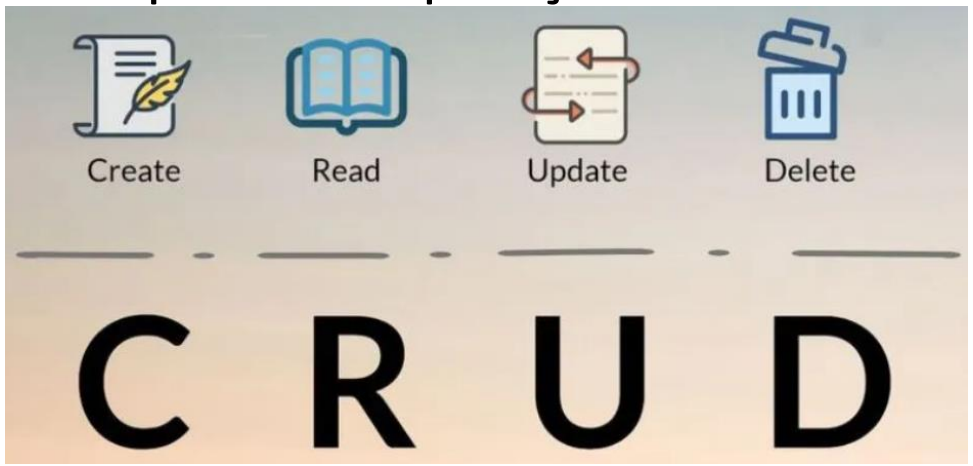
Projeto de Banco de Dados



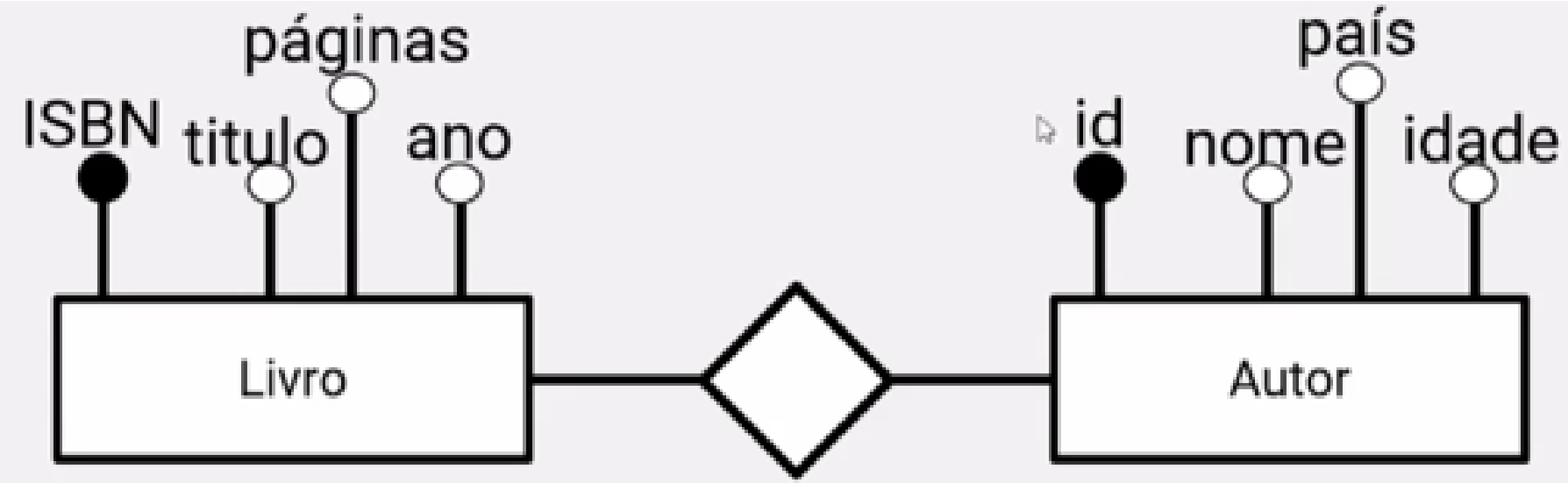
Estudo de Caso Escola – Requisitos Funcionais

- RF01 – Manter dados do professor:
  - O sistema deve permitir a inclusão, alteração, exclusão e consulta dos dados dos professores. Sendo que estas operações serão realizadas pela secretaria.

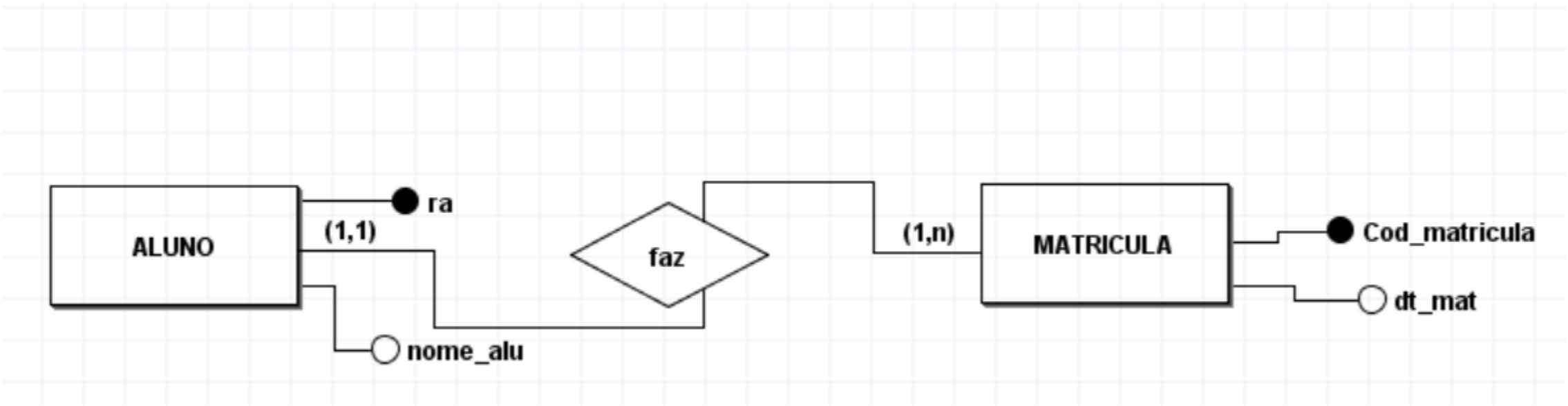
- RF02 – Manter dados do aluno:
  - O sistema deve permitir a inclusão, alteração, exclusão e consulta dos dados dos alunos. Sendo que estas operações serão realizadas pela secretaria.



Modelo Conceitual – Pequeno exemplo

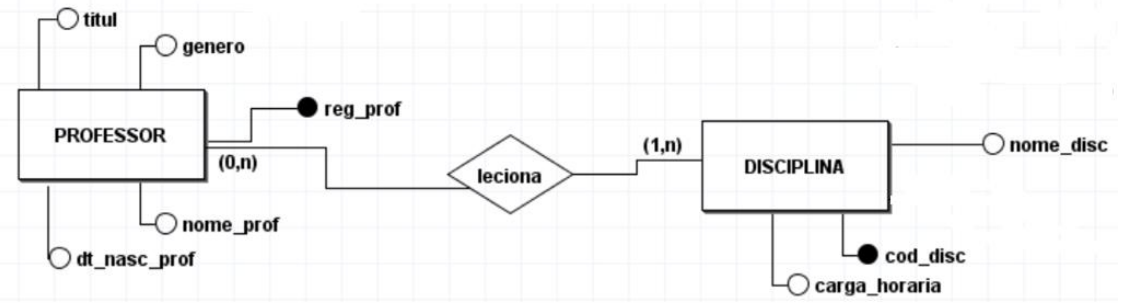


Exemplo

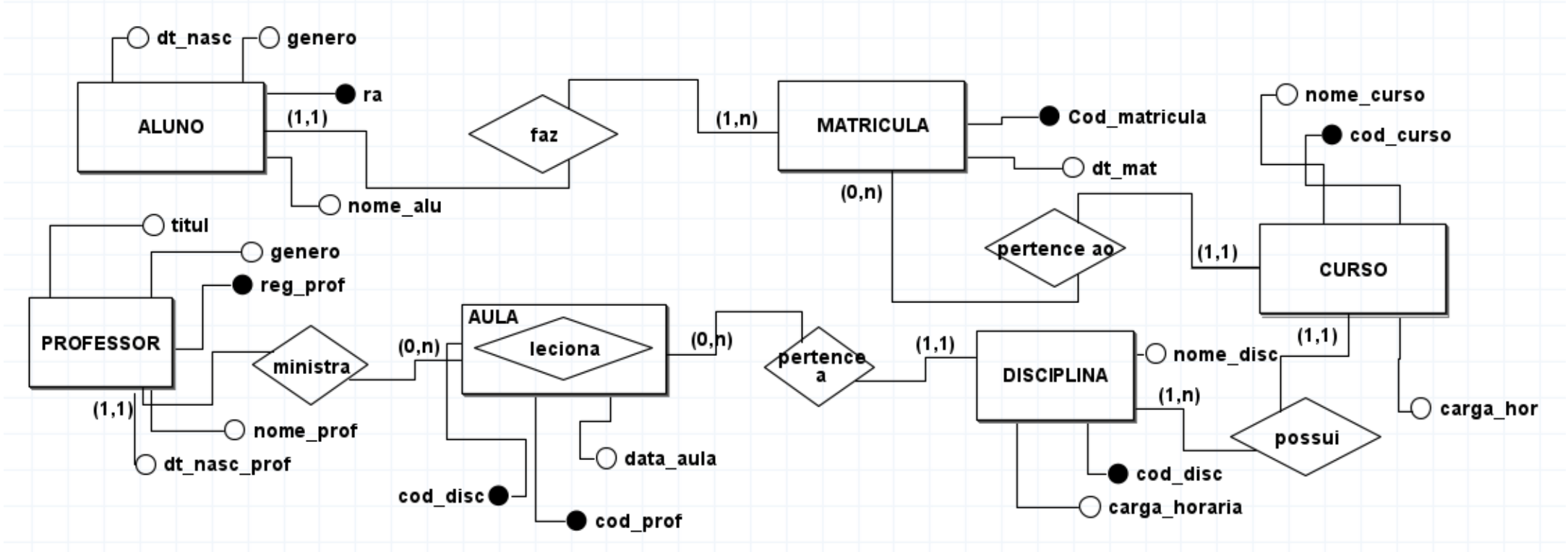


Um aluno faz no mínimo 1 matrícula e no máximo n matrículas.
Uma matrícula é de no mínimo 1 aluno e no máximo 1 aluno.

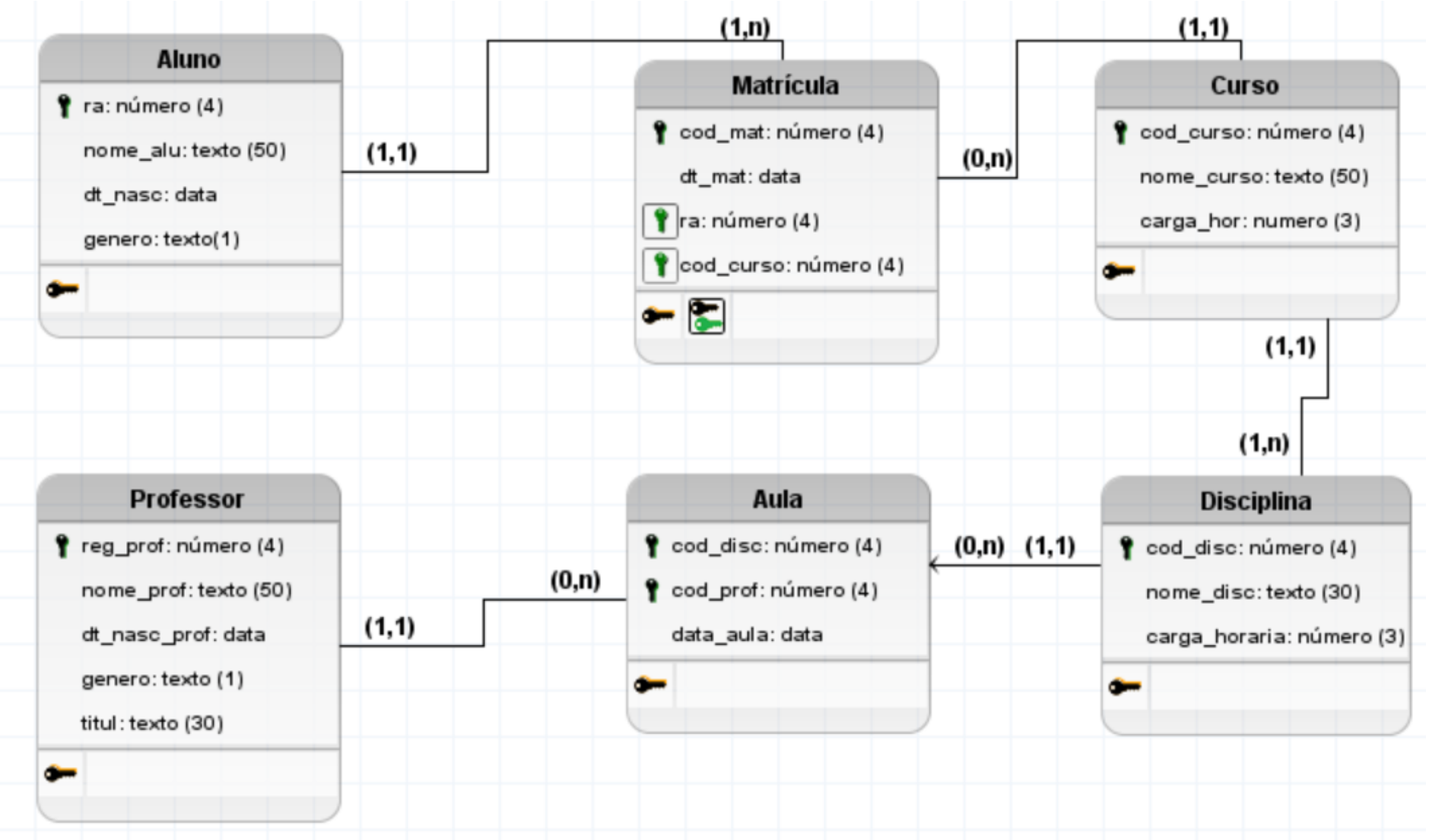
Entidade Associativa



- Note que o relacionamento entre as entidades DISCIPLINA e PROFESSOR é do tipo N:N, isso significa, que será necessário fazer um ajuste no modelo conceitual com a criação da entidade associativa.



Possível solução



Formas Normais

A normalização segue um processo gradual através de "formas normais", cada uma com regras específicas que constroem sobre a anterior. Na prática, a maioria das aplicações utiliza até a Terceira Forma Normal (3NF), que oferece um excelente equilíbrio entre organização e simplicidade.

Primeira Forma Normal (1NF)

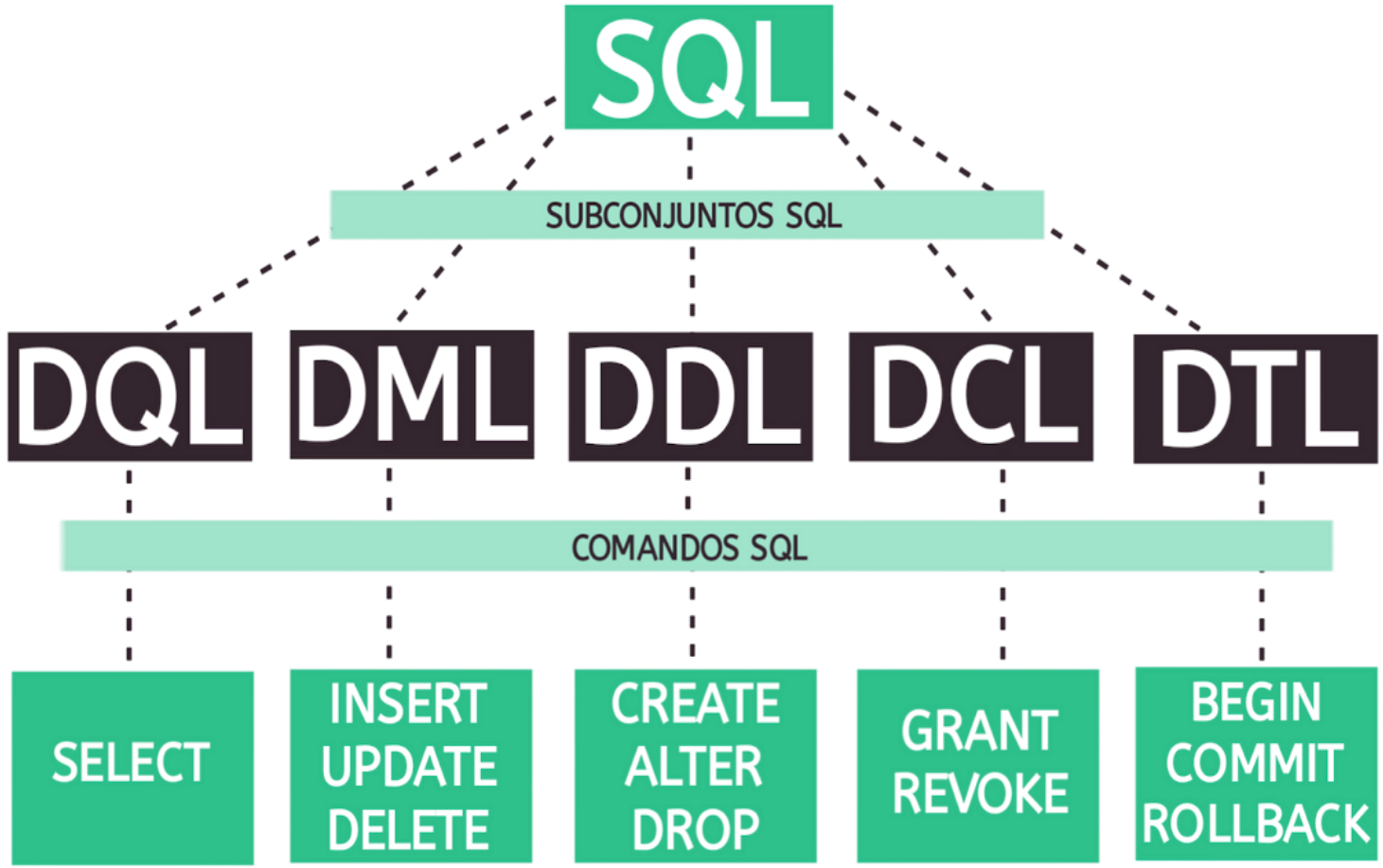
Elimina grupos repetitivos e garante que cada campo contenha apenas um valor atômico, indivisível

Segunda Forma Normal (2NF)

Remove dependências parciais, assegurando que todos os dados não-chave dependem completamente da chave primária inteira

Terceira Forma Normal (3NF)

Elimina dependências transitivas, garantindo que dados não-chave não dependem de outros dados não-chave


SQL
SUBCONJUNTOS SQL
DQL
DML
DDL
DCL
DTL
COMANDOS SQL
SELECT
INSERT
UPDATE
DELETE
CREATE
ALTER
DROP
GRANT
REVOKE
BEGIN
COMMIT
ROLLBACK

Instruções SQL

- Instruções SQL não têm distinção entre maiúsculas e minúsculas.
- Instruções SQL podem estar em uma ou mais linhas.
- Palavras-chave não podem ser abreviadas ou divididas entre as linhas.
- Normalmente, as cláusulas são colocadas em linhas separadas.
- Guias e indentações são usadas para aperfeiçoar a legibilidade.

Criação de tabelas – Regras de Nomenclatura

- Deve começar com uma letra;
- Pode ter de 1 a 128 caracteres;
- Deve conter somante A-Z, a-z, _, $ e #;
- Não deve duplicar o nome de outro objeto de propriedade do mesmo usuário;
- Não deve ser uma palavra reservada.

Tipos de dados

Categoria	Tipo	Descrição	Exemplo de Uso
Numéricos	INT	Números inteiros.	Idade, Quantidade, IDs.
	DECIMAL(M,D)	Números decimais exatos.	Preços, Salários, Peso.
	FLOAT / DOUBLE	Números de ponto flutuante (aproximados).	Coordenadas GPS, Médias científicas.
Strings (Texto)	VARCHAR(N)	Texto de tamanho variável (até N caracteres).	Nomes, E-mails, Logradouros.
	CHAR(N)	Texto de tamanho fixo.	Siglas de Estados (UF), Gênero (M/F).
	TEXT	Grandes blocos de texto.	Descrições de produtos, Artigos.
Data e Hora	DATE	Data no formato AAAA-MM-DD.	Data de nascimento.
	DATETIME	Data e hora combinadas.	Registro de Logs, Data da venda.
	TIME	Apenas o horário.	Horário de abertura.
Lógicos	BOOLEAN	Verdadeiro ou Falso (Internamente é um TINYINT).	Ativo/Inativo, Pago/Pendente.

Para SGBD Oracle:

Varchar: armazena até 2000 caracteres.

Varchar2: dobra a capacidade (armazena até 4000 caracteres)

Comandos Iniciais SQL

CREATE (CRIAR)

O comando CREATE é utilizado para criar objetos no banco de dados, como: Bancos de dados, Tabelas, Usuários, Visões (views), Procedimentos armazenados, entre outros.

É como construir a fundação de um sistema: definimos onde os dados serão guardados, o que será armazenado, e quem poderá interagir com essas informações.

Sintaxe básica do comando CREATE DATABASE

CREATE DATABASE nome do banco de dados que deverá ser criado**;**



```
CREATE DATABASE BibliotecaDigital;
```

Comandos Iniciais SQL

Sintaxe Básica comando CREATE TABLE

CREATE TABLE nome_da_tabela (

nome_coluna1 tipo_dado1 [restrições],

nome_coluna2 tipo_dado2 [restrições],

...

[restrições de tabela]

);

```sql
CREATE TABLE Livros (
    id INT AUTO_INCREMENT PRIMARY KEY,
    titulo VARCHAR(100),
    autor VARCHAR(80),
    ano_publicacao INT,
    genero VARCHAR(50)
);
```

Componentes explicados:

- `nome_da_tabela` : é o nome da tabela que você quer criar.
- `nome_coluna` : nome de cada campo da tabela.
- `tipo_dado` : define o tipo de informação (como `INT`, `VARCHAR`, `DATE`, etc.).
- `restrições` *(opcional)*: regras como `PRIMARY KEY`, `NOT NULL`, `UNIQUE`, etc.

Criando uma tabela

SQL Worksheet

```
1  create table Aluno (
2      ra number(5),
3      nome varchar(100),
4      data_matricula date
5  );
6
7  create table disciplina(
8      cod_disc number(5) primary key,
9      nome_disc varchar2(50) not null,
10     carga_horaria number(3)
11 );
```

Restrições (constraints)

- Impõem regras que podem ser no nível da coluna ou no nível da tabela:
- NOT NULL → Não aceita valor nulo
- UNIQUE → Valor deve ser único
- PRIMARY KEY → Chave primária (não nulo e único)
- FOREIGN KEY → Chave estrangeira (deve ter valor correspondente na tabela mãe)
- CHECK → Define uma condição de valor para a coluna

A Restrição CHECK

- Define uma condição que cada linha deve satisfazer

```
..., deptno   NUMBER(2),
      CONSTRAINT emp_deptno_ck
            CHECK (DEPTNO BETWEEN 10 AND 99),...
```

Exemplo

```
create table professor (
    cod_prof number(5) primary key,
    nome_prof varchar2(100) not null,
    dt_nasc date,
    sexo char(1) check (sexo='M' or sexo='F' or sexo='O'),
    cpf varchar2(11) unique,
    cod_disc number(5) references disciplina(cod_disc)
);
describe professor;
```

A instrução ALTER TABLE

- Use a instrução ALTER TABLE para:
  - Adicionar uma nova coluna
  - Modificar uma coluna existente
  - Definir um valor default para a nova coluna

```
ALTER TABLE tabela
ADD         (tipo de dados da coluna [DEFAULT expr]
            [, tipo de dados da coluna ]...);
```

```
ALTER TABLE tabela
MODIFY      (tipo de dados da coluna [DEFAULT expr]
            [, tipo de dados da coluna ]...);
```

Exemplos de ALTER TABLE

```
-- Alterações na tabela:
-- Incluindo coluna na tabela:
alter table aluno
add data_nasc date;

-- Alterando coluna:
alter table aluno
modify nome varchar2(150) not null;


-- Renomeando uma coluna
alter table aluno
rename column data_nasc to data_nascimento;
```

Adicionando constraint

```
-- Adicionando constraint:
alter table aluno
add constraint pk_alu primary key (ra);
```

Exclusão de coluna e de tabela

```
-- Renomeando e excluindo tabela:
rename aluno to alu;

drop table alu;
```

Exclusão de coluna

```
-- Exclusão de coluna:
alter table aluno
drop column data_nasc;
```

🔧 Principais comandos DDL:

Comando	Função
`CREATE`	Cria novas estruturas no banco (tabelas, bancos, índices)
`ALTER`	Modifica uma estrutura existente (como adicionar colunas)
`DROP`	Remove completamente a estrutura (ex: apagar tabela)
`TRUNCATE`	Apaga todos os dados de uma tabela, mas **mantém sua estrutura**

📌 Exemplos práticos:

🛠️ Criar uma tabela:

```sql
CREATE TABLE clientes (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nome VARCHAR(100),
    email VARCHAR(100)
);
```

🧱 Alterar a tabela:

```sql
ALTER TABLE clientes ADD telefone VARCHAR(20);
```

🗑️ Apagar a tabela:

```sql
DROP TABLE clientes;
```

🧹 Apagar todos os dados (sem deletar a tabela):

```sql
TRUNCATE TABLE clientes;
```

Criar as tabelas a seguir e incluir alguns registros

